

Traccia di correzione — Seconda prova intermedia

Punteggio totale: **36 punti** (es. 1–4 (teoria): 10 pt · es. 5–6 (analisi del codice): 10 pt · es. 7–8 (programmazione): 16 pt)

Esercizio 1 — Domanda aperta (4 punti)

Variante A — Definizione vs chiamata; funzioni pure vs void

Risposta attesa: - **Definizione:** stabilisce il comportamento della funzione. Il codice nel corpo non viene eseguito immediatamente; la definizione associa un nome a un blocco di istruzioni con parametri formali. - **Chiamata (invocazione):** fa eseguire concretamente il corpo della funzione, sostituendo i parametri formali con gli argomenti reali passati. - **Funzione pura (o produttiva):** usa `return` per restituire un valore al chiamante; può essere usata in un'espressione (es. `x = f(3)`). - **Funzione void:** non restituisce un valore utile (nessun `return` o `return None` implicito); il suo effetto è sulle strutture dati passate o sull'output a schermo.

Criteri di punteggio:

Punti	Criterio
4	Tutti e quattro i concetti (definizione, chiamata, pura, void) spiegati correttamente
3	Tre concetti corretti, uno impreciso o mancante
2	Due concetti corretti
1	Solo un concetto corretto o risposta molto parziale
0	Risposta errata o assente

Variante B — Scope di una variabile

Risposta attesa: - Lo **scope** (ambito) di una variabile definisce in quale parte del programma quella variabile è visibile e utilizzabile. - Le variabili definite dentro una funzione hanno **scope locale**: esistono solo durante l'esecuzione della funzione e sono invisibili al di fuori. - Le variabili definite nel programma principale hanno **scope globale**: sono visibili in tutto il programma, ma non modificabili direttamente dall'interno di una funzione. - Conseguenza dell'indipendenza: una funzione può ricevere dati solo tramite parametri e restituirli solo tramite `return`, rendendo il comportamento isolato e prevedibile.

Criteri di punteggio:

Punti	Criterio
4	Definizione di scope corretta + distinzione locale/globale + almeno una conseguenza concreta
3	Definizione corretta e distinzione locale/globale, ma conseguenze assenti o imprecise
2	Intuizione corretta ma spiegazione molto superficiale
1	Risposta molto parziale con almeno un'idea corretta
0	Risposta errata o assente

Esercizio 2 — Scelta multipla (2 punti)

Variante A — `a = [1, 2]`, poi `b = a`, poi `b.append(3)`

`b = a` non crea una copia: entrambe le variabili puntano allo stesso oggetto lista in memoria. Modificare la lista tramite `b` la modifica anche per `a`.

- Solo la lista `b` conterrà il numero 3. — errata: `a` e `b` sono alias dello stesso oggetto
- Viene creata automaticamente una copia di `a` prima della modifica. — errata: Python non copia automaticamente
- Entrambe le variabili `a` e `b` punteranno alla lista aggiornata. **(RISPOSTA CORRETTA)**
- Python genera un errore perché non è possibile avere due nomi per lo stesso oggetto. — errata: l'aliasing è perfettamente legale

Variante B — `a = [1, 2]`, poi `b = [1, 2]`, poi `b.append(3)`

`b = [1, 2]` crea un nuovo oggetto lista, indipendente da `a`. Modificare `b` non ha effetto su `a`.

- Solo la lista `b` conterrà il numero 3. **(RISPOSTA CORRETTA)**
- Viene creata automaticamente una copia di `a` prima della modifica. — errata: `b` è già un oggetto separato, non una copia di `a`
- Entrambe le variabili `a` e `b` punteranno alla lista aggiornata. — errata: sono oggetti distinti
- Python genera un errore perché non è possibile avere due nomi per lo stesso oggetto. — errata

Punteggio: **2 punti** se corretta, **0 punti** se errata (nessun punto parziale).

Esercizio 3 — Scelta multipla (2 punti)

Variante A — Quale tipo è **immutabile**? → **Int**

Un intero non può essere modificato in-place: `x = x + 1` crea un nuovo oggetto intero e riassegna `x`, non modifica l'oggetto originale.

- List — errata: le liste sono mutabili (append, assegnazione per indice, ecc.)
- Int **(RISPOSTA CORRETTA)**
- Entrambi i tipi menzionati — errata
- Nessuno dei tipi menzionati — errata

Variante B — Quale tipo è **mutabile**? → **List**

Una lista può essere modificata in-place (es. `lista.append(x)`, `lista[i] = v`) senza creare un nuovo oggetto.

- List **(RISPOSTA CORRETTA)**
- Int — errata: gli interi sono immutabili
- Entrambi i tipi menzionati — errata
- Nessuno dei tipi menzionati — errata

Punteggio: **2 punti** se corretta, **0 punti** se errata.

Esercizio 4 — Scelta multipla (2 punti)

Variante A — Cosa stampa il codice con `f(x): x = x + 1` e `a = 5`?

```
def f(x):
    x = x + 1

a = 5
f(a)
print(a)
```

`x = x + 1` riassegna la variabile locale `x` dentro `f`: non modifica `a`, perché gli interi sono immutabili e `x` è una copia locale del valore.

- 5 (**RISPOSTA CORRETTA**)
- 6 — errata: la riassegnazione di `x` non si propaga fuori dalla funzione
- `None` — errata: `print(a)` stampa il valore di `a`, non il valore di ritorno di `f`
- Python genera un errore perché `x` non è definita fuori da `f` — errata: non si accede a `x` fuori da `f`

Variante B — Cosa stampa il codice con `f(lista): lista.append(99)` e `a = [1, 2, 3]`?

```
def f(lista):
    lista.append(99)

a = [1, 2, 3]
f(a)
print(a)
```

`lista` punta allo stesso oggetto lista di `a`. Le liste sono mutabili: `lista.append(99)` modifica l'oggetto condiviso, quindi `a` risulta modificata.

- `[1, 2, 3]` — errata: la lista è stata modificata in-place
- `[1, 2, 3, 99]` (**RISPOSTA CORRETTA**)
- `None` — errata
- Python genera un errore perché `lista` non è definita fuori da `f` — errata

Punteggio: **2 punti** se corretta, **0 punti** se errata.

Esercizio 5 — Analisi del codice (6 punti)

Variante A — Elementi minori precedenti

Il programma termina senza errori di esecuzione. L'output **non è quello atteso**.

Output atteso: `[0, 0, 2, 1]` **Output effettivo:** `[2, 0, 3, 1]`

Errore logico: il ciclo interno usa `while j < len(lista)` invece di `while j < i`. In questo modo, per ogni elemento in posizione `i`, si confronta con **tutti** gli elementi della lista (anche quelli che lo seguono), invece di guardare solo quelli che lo **precedono** (posizioni da 0 a `i-1`).

Dimostrazione per `i=0` (`lista[0]=3`):

Con il bug (`j < len(lista)`), il ciclo interno esegue 4 iterazioni e conta `lista[1]=1` e `lista[3]=2` come “minori che precedono” — ma sono elementi che vengono *dopo*, non prima.

i	j	j < len(lista)	lista[j] < lista[i]	contatore
0	0	True	3 < 3 → False	0
0	1	True	1 < 3 → True	1

i	j	j < len(lista)	lista[j] < lista[i]	contatore
0	2	True	4 < 3 → False	1
0	3	True	2 < 3 → True	2

Con il codice corretto ($j < i = j < 0$), il ciclo non esegue nemmeno una volta → contatore=0.

Codice corretto:

```
lista = [3, 1, 4, 2]

contatori = []
i = 0
while i < len(lista):
    contatore = 0
    j = 0
    while j < i:
        if lista[j] < lista[i]:
            contatore = contatore + 1
        j = j + 1
    contatori.append(contatore)
    i = i + 1

print(contatori)
```

Variante B — Elementi maggiori precedenti

Il programma termina senza errori di esecuzione. L'output **non è quello atteso**.

Output atteso: [0, 1, 0, 2] **Output effettivo:** [1, 3, 0, 2]

Errore logico: identico alla variante A — il ciclo interno usa `while j < len(lista)` invece di `while j < i`, quindi conta anche gli elementi che vengono *dopo* la posizione i.

Dimostrazione per i=0 (lista[0]=3):

Con il bug, il ciclo conta lista[2]=4 come “maggiore che precede” — ma la posizione 2 viene dopo, non prima.

i	j	j < len(lista)	lista[j] > lista[i]	contatore
0	0	True	3 > 3 → False	0
0	1	True	1 > 3 → False	0
0	2	True	4 > 3 → True	1
0	3	True	2 > 3 → False	1

Con il codice corretto ($j < i = j < 0$), il ciclo non esegue → contatore=0.

Codice corretto:

```
lista = [3, 1, 4, 2]
```

```

contatori = []
i = 0
while i < len(lista):
    contatore = 0
    j = 0
    while j < i:
        if lista[j] > lista[i]:
            contatore = contatore + 1
        j = j + 1
    contatori.append(contatore)
    i = i + 1

print(contatori)

```

Criteri di punteggio (6 punti = 3 + 3):

Punti	Prima parte — Identificazione dell'errore
3	Identifica il bound sbagliato (<code>len(lista)</code> invece di <code>i</code>), spiega che il ciclo interno guarda anche gli elementi successivi e lo dimostra con la tabella
2	Identifica che l'output è sbagliato e individua la riga errata
1	Identifica che l'output è sbagliato ma la motivazione è imprecisa o mancante
0	Risponde che il codice funziona correttamente

Punti	Seconda parte — Codice corretto
3	Codice corretto: <code>while j < i</code> con la condizione giusta
2	Logica corretta ma piccolo errore (es. <code>j < i - 1</code> , salta l'elemento immediatamente precedente)
1	Soluzione parziale: intuisce il problema del bound ma implementazione incompleta
0	Codice non presentato o completamente errato

Esercizio 6 — Tabella di tracciamento (4 punti)

Variante A — Soluzione completa

```

def incrementa(lista, k):
    i = 0
    while i < len(lista):
        lista[i] = lista[i] + k
        i = i + 1

a = [-2, 5, 0]
incrementa(a, 10)
print(a)

```

Passo	Codice sorgente	Stato - globale	Stato - funzione	Output
1	<code>a = [-2, 5, 0]</code>	<code>a->[-2, 5, 0]</code>		
2	<code>chiama incrementa(a, 10)</code>	<code>a->[-2, 5, 0]</code>	<code>lista->a->[-2,5,0], k->10</code>	
3	<code>i = 0</code>	<code>a->[-2, 5, 0]</code>	<code>lista->[-2,5,0], k->10, i->0</code>	
4	<code>while i < len(lista): guardia True</code>	<code>a->[-2, 5, 0]</code>	<code>lista->[-2,5,0], k->10, i->0</code>	
5	<code>lista[0] = lista[0] + 10</code>	<code>a->[8, 5, 0]</code>	<code>lista->[8,5,0], k->10, i->0</code>	
6	<code>i = i + 1</code>	<code>a->[8, 5, 0]</code>	<code>lista->[8,5,0], k->10, i->1</code>	
7	<code>while i < len(lista): guardia True</code>	<code>a->[8, 5, 0]</code>	<code>lista->[8,5,0], k->10, i->1</code>	
8	<code>lista[1] = lista[1] + 10</code>	<code>a->[8, 15, 0]</code>	<code>lista->[8,15,0], k->10, i->1</code>	
9	<code>i = i + 1</code>	<code>a->[8, 15, 0]</code>	<code>lista->[8,15,0], k->10, i->2</code>	
10	<code>while i < len(lista): guardia True</code>	<code>a->[8, 15, 0]</code>	<code>lista->[8,15,0], k->10, i->2</code>	
11	<code>lista[2] = lista[2] + 10</code>	<code>a->[8, 15, 10]</code>	<code>lista->[8,15,10], k->10, i->2</code>	
12	<code>i = i + 1</code>	<code>a->[8, 15, 10]</code>	<code>lista->[8,15,10], k->10, i->3</code>	
13	<code>while i < len(lista): guardia False; ritorna; print(a)</code>	<code>a->[8, 15, 10]</code>	<code>lista->[8,15,10], k->10, i->3</code>	<code>[8, 15, 10]</code>

Variante B — Soluzione completa

```

def azzerare(lista):
    i = 0
    c = 0
    while i < len(lista):
        if lista[i] < 0:
            c = c + 1
            lista[i] = 0
            i = i + 1

    lista.append(c)

numeri = [5, 10, -1]
azzerare(numeri)
print(numeri)

```

Passo	Codice sorgente	Stato - globale	Stato - funzione	Output
1	<code>numeri = [5, 10, -1]</code>	<code>numeri->[5, 10, -1]</code>		
2	<code>chiama azzera(numeri)</code>	<code>numeri->[5, 10, -1]</code>	<code>lista->numeri->[5, 10, -1]</code>	
3	<code>i = 0, c = 0</code>	<code>numeri->[5, 10, -1]</code>	<code>lista->[5,10,-1], i->0, c->0</code>	
4	<code>while i < len(lista): guardia True</code>	<code>numeri->[5, 10, -1]</code>	<code>lista->[5,10,-1], i->0, c->0</code>	
5	<code>if lista[0] < 0: guardia False (5 0)</code>	<code>numeri->[5, 10, -1]</code>	<code>lista->[5,10,-1], i->0, c->0</code>	
6	<code>lista[0] = 0</code>	<code>numeri->[0, 10, -1]</code>	<code>lista->[0,10,-1], i->0, c->0</code>	
7	<code>i = i + 1</code>	<code>numeri->[0, 10, -1]</code>	<code>lista->[0,10,-1], i->1, c->0</code>	
8	<code>while i < len(lista): guardia True; if lista[1] < 0: guardia False</code>	<code>numeri->[0, 10, -1]</code>	<code>lista->[0,10,-1], i->1, c->0</code>	
9	<code>lista[1] = 0</code>	<code>numeri->[0, 0, -1]</code>	<code>lista->[0,0,-1], i->1, c->0</code>	
10	<code>i = i + 1</code>	<code>numeri->[0, 0, -1]</code>	<code>lista->[0,0,-1], i->2, c->0</code>	
11	<code>while i < len(lista): guardia True; if lista[2] < 0: guardia True</code>	<code>numeri->[0, 0, -1]</code>	<code>lista->[0,0,-1], i->2, c->0</code>	
12	<code>c = c + 1</code>	<code>numeri->[0, 0, -1]</code>	<code>lista->[0,0,-1], i->2, c->1</code>	
13	<code>lista[2] = 0</code>	<code>numeri->[0, 0, 0]</code>	<code>lista->[0,0,0], i->2, c->1</code>	
14	<code>i = i + 1; while i < len(lista): guardia False</code>	<code>numeri->[0, 0, 0]</code>	<code>lista->[0,0,0], i->3, c->1</code>	
15	<code>lista.append(1); ritorna; print(numeri)</code>	<code>numeri->[0, 0, 1]</code>		<code>[0, 0, 0, 1]</code>

Criteri di punteggio (4 punti per le righe da completare):

Punti	Criterio
4	Tabella completa e corretta (al massimo 1 errore di calcolo minore)
3	1-2 righe sbagliate, ma il flusso del ciclo e gli effetti sulla lista condivisa sono corretti
2	Flusso parzialmente corretto (es. prima iterazione corretta, poi errori sistematici)
1	Tracciamento presente ma con errori significativi; almeno l'output finale è corretto
0	Tabella non completata o completamente errata

Esercizio 7 — Programmazione (6 punti)

Variante A — `filtra_magiori(lista, soglia)`

Soluzione attesa:

```
def filtra_magiori(lista, soglia):
    risultato = []
    i = 0
    while i < len(lista):
        if lista[i] > soglia:
            risultato.append(lista[i])
        i = i + 1
    return risultato
```

Con `lista = [7, 4, -1, 9]` e `soglia = 0`: output `[7, 4, 9]` (`-1` è escluso perché non strettamente maggiore di `0`).

Variante B — `filtra_lunghe(lista, n)`

Soluzione attesa:

```
def filtra_lunghe(lista, n):
    risultato = []
    i = 0
    while i < len(lista):
        if len(lista[i]) > n:
            risultato.append(lista[i])
        i = i + 1
    return risultato
```

Con `lista = ["ciao", "sì", "informatica", "no"]` e `n = 3`: output `["ciao", "informatica"]` (`len("ciao")=4 > 3`, `len("informatica")=11 > 3`; “sì” e “no” esclusi).

Criteri di punteggio (6 punti):

Punti	Criterio
6	Funzione corretta: nuova lista, condizione giusta (<code>></code>), <code>return</code> presente, non modifica la lista originale
5	Piccolo errore (es. <code>>=</code> invece di <code>></code> , oppure manca il <code>return</code> ma il resto è corretto)
4	Struttura corretta ma la funzione modifica la lista originale invece di crearne una nuova
3	Ciclo e raccolta corretti ma la condizione di filtro è sbagliata
2	Intuizione corretta (ciclo + <code>if</code> + <code>append</code>) ma implementazione non funzionante
1	Idea corretta ma molto incompleta
0	Soluzione assente o completamente errata

Errori comuni e come pesarli: - Usa `>=` invece di `>` (non strettamente maggiore): `-1` pt - Manca `return` (restituisce `None`): `-1` pt - Modifica `lista` invece di creare `risultato`: `-2` pt

Esercizio 8 — Programmazione avanzata (10 punti)

Variante A — `ripeti(lista, k)`

Soluzione attesa:

```
def ripeti(lista, k):
    risultato = []
    i = 0
    while i < len(lista):
        j = 0
        while j < k:
            risultato.append(lista[i])
            j = j + 1
        i = i + 1
    return risultato
```

```
k = int(input())
while k <= 0:
    k = int(input())
```

```
numeri = []
i = 0
while i < 4:
    numeri.append(int(input()))
    i = i + 1
```

```
print(ripeti(numeri, k))
print(ripeti(numeri, k + 1))
```

Con `k = 2` e `numeri = [5, 3, 8, 1]`:

```
[5, 5, 3, 3, 8, 8, 1, 1]
[5, 5, 5, 3, 3, 3, 8, 8, 8, 1, 1, 1]
```

Variante B — `intercala(lista, k)`

Soluzione attesa:

```
def intercala(lista, k):
    risultato = []
    i = 0
    while i < len(lista):
        risultato.append(lista[i])
        j = 0
        while j < k:
            risultato.append(0)
            j = j + 1
        i = i + 1
    return risultato
```

```
k = int(input())
while k <= 0:
```

```

k = int(input())

numeri = []
i = 0
while i < 4:
    numeri.append(int(input()))
    i = i + 1

print(intercala(numeri, k))
print(intercala(numeri, k + 1))

```

Con $k = 1$ e $\text{numeri} = [5, 3, 8, 1]$:

$[5, 0, 3, 0, 8, 0, 1, 0]$

$[5, 0, 0, 3, 0, 0, 8, 0, 0, 1, 0, 0]$

Criteri di punteggio (10 punti = 3 + 7):

Punti	Parte 1 — Input e validazione (3 pt)
1	Lettura corretta di k come intero
1	Ciclo di validazione (<code>while k <= 0</code>) corretto
1	Lettura dei 4 numeri interi in una lista

Punti	Parte 2 — Funzione e output (7 pt)
2	Ciclo esterno che scorre la lista originale
3	Ciclo interno corretto (A: ripete l'elemento k volte; B: aggiunge k zeri dopo l'elemento)
1	<code>return risultato</code> presente e corretto
1	Due chiamate a stampa: con k e con $k + 1$